Taxi Service — Backend Requirements

Service Position, Integration Findings, Required Behaviour and Delivery Sequence

1. Purpose and Scope

To implement `services/driver-taxi-service` so customers can book a car ride and taxi drivers can receive, run and complete that ride, without moving booking, matching or trip ownership into any other service.

This document defines the required backend behaviour, data model, endpoint contracts, integration constraints and delivery sequence for the service. It covers the backend only. Flutter application work for `apps/taxi_provider` and the customer app is out of scope.

2. Existing Service Position

- `services/driver-taxi-service` exists as a scaffold. `cmd/main.go` calls `serviceapp.Run` with capability strings only.
- `internal/config/config.go` defines `AppEnv`, `HTTPAddr`, `DatabaseURL` and `Redis` only.
- `internal/features/{auth,providers,vehicles,rides,trips,matching}/` contain README placeholders and no code.
- There is no `migrations/` directory, no database pool, no `.env`, and no bootstrap script.
- The Go module path is `cosmicforge/logistics/services/taxi-service`. The API base is `/api/v1/taxi`. The port is `8102`. These are fixed and must not be renamed.
- `services/driver-hauling-service` is the reference implementation. It owns a full lifecycle in one service, and its matching engine has already had its concurrency defects corrected.
- `services/driver-dispatch-delivery-service` splits booking, trip and broadcast state across three tables and an asynq queue because it also serves an external partner API. The taxi service has no partner API and must not adopt that structure.

3. Platform Integration Findings Requiring Closure

The platform was provisioned with taxi slots when other services were built. The following were identified by inspection of the running services and must be closed before feature work begins. Each one fails silently at runtime.

Correct already, no action required:

- `payment-wallet-service` accepts provider type `taxi` with token secret `development-taxi-access-token-secret`. Driver earnings, bank accounts and withdrawals require no changes in that service.
- `payment-wallet-service` trusts HMAC service identity `taxi-service` with secret `development-payment-wallet-service-secret`. The hauling service uses a different secret for the same purpose; that value must not be copied.
- `media-file-service` trusts `taxi-service=development-media-token`. Document and photo uploads require no changes.
- `support-dispute-service` trusts `taxi-service=development-support-dispute-service-secret` for inbound service calls.

Requiring closure:

#	Finding	Required closure
3.1	`support-dispute-service` expects provider token secret `development-taxi-provider-token-secret`. `payment-wallet-service` expects `development-taxi-access-token-secret` for the same tokens. One token cannot satisfy both.	Align `SUPPORT_DISPUTE_TAXI_PROVIDER_TOKEN_SECRET` to `development-taxi-access-token-secret` in `services/support-dispute-service/internal/config/config.go`, `.env` and `.env.example`.
3.2	`NOTIFICATION_SERVICE_SECRETS` lists customer, hauling, payment-wallet, dispatch and support-dispute. Taxi is absent. Every notification the taxi service sends returns 401.	Add `driver-taxi-service=development-taxi-notification-secret` to `services/notification-service/.env`.
3.3	`SUPPORT_DISPUTE_TAXI_SERVICE_URL` is blank. Driver complaints are filed without the driver's name attached.	Implement `GET /internal/providers/:id` (section 11.7), then set the URL to `http://localhost:8102` and the secret to `development-support-dispute-service-secret`.
3.4	No local bootstrap script exists for taxi.	Create `scripts/bootstrap_services/taxi-local-bootstrap.sh` from the hauling equivalent, using Postgres `5434`, Redis `6381`, database `taxi_service`. Register it in `bootstrap-all.sh`.
3.5	The `taxi-postgres`, `taxi-redis` and `taxi-service` blocks in `infra/docker-compose.yml` are commented out.	Uncomment when the service is containerised.

Intended result

The taxi service authenticates, charges, notifies and reports through the existing platform services without modification to those services beyond items 3.1 and 3.2.

4. Service Configuration

Extend `internal/config/config.go` to the hauling shape. Internal client URLs must be bare origins; `shared/go/walletclient` and `shared/go/mediaclient` append their own paths, and a prefixed URL returns 404 on every call.

```
APP_ENV=development
HTTP_ADDR=:8102
TAXI_MIGRATION=true

TAXI_DATABASE_URL=postgres://cosmicforge_logistics:cosmicforge_logistics@localhost:5434/taxi_service?ssl
    mode=disable
TAXI_REDIS_ADDR=localhost:6381
TAXI_REDIS_PASSWORD=
TAXI_REDIS_DB=0

TAXI_PROVIDER_TOKEN_SECRET=development-taxi-access-token-secret
TAXI_PROVIDER_REFRESH_SECRET=development-taxi-provider-refresh-secret
TAXI_PROVIDER_OTP_SECRET=development-taxi-provider-otp-secret
TAXI_CUSTOMER_TOKEN_SECRET=development-customer-access-token-secret

TAXI_ACCESS_TOKEN_TTL_SECONDS=3600
TAXI_REFRESH_TOKEN_TTL_DAYS=30
TAXI_OTP_TTL_SECONDS=600
TAXI_OTP_RATE_WINDOW_SECONDS=60
TAXI_OTP_MAX_REQUESTS=5
TAXI_OTP_MAX_ATTEMPTS=5
TAXI_OTP_DEBUG=true

TAXI_NOTIFICATION_URL=http://localhost:8106
TAXI_NOTIFICATION_SECRET=development-taxi-notification-secret
TAXI_PAYMENT_URL=http://localhost:8105
TAXI_PAYMENT_SECRET=development-payment-wallet-service-secret

TAXI_SERVICE_SECRETS=support-dispute-service=development-support-dispute-service-secret
```

```
TAXI_BOOKING_MATCH_TIMEOUT=30
TAXI_BOOKING_SEARCH_WINDOW=60
TAXI_PROVIDER_ONLINE_TTL=90
TAXI_MATCH_MAX_RADIUS_KM=10
```

Requirements:

- `TAXI_PROVIDER_TOKEN_SECRET` must equal the payment-wallet value in section 3. Changing it breaks driver earnings and withdrawals.
- `TAXI_CUSTOMER_TOKEN_SECRET` must equal `CUSTOMER_ACCESS_TOKEN_SECRET` in customer-service.
- `TAXI_PROVIDER_ONLINE_TTL` must remain at 90 seconds. A longer value keeps force-quit drivers in the online set and lets them consume the entire search window.
- When `TAXI_PAYMENT_URL` or `TAXI_PAYMENT_SECRET` is unset, the payment client is nil and every payment step is a no-op. The same applies to notifications. Local development runs without payment-wallet or notification-service.
- Provider tokens carry role `taxi_provider` and service `taxi`. Both values are already assumed by `support-dispute-service` and `payment-wallet-service`.

5. Data Model

Create `migrations/001_taxi_core.sql`. Migrations are service-local and applied in sorted filename order when `TAXI_MIGRATION=true`. No cross-service foreign keys are permitted; `customer_id` is a bare UUID owned by customer-service.

```
CREATE TABLE IF NOT EXISTS taxi_providers (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    phone             TEXT UNIQUE,
    email             TEXT UNIQUE,
    first_name        TEXT NOT NULL DEFAULT '',
    last_name         TEXT NOT NULL DEFAULT '',
    profile_photo_url TEXT,
    photo_asset_id    TEXT,
    status            TEXT NOT NULL DEFAULT 'active',
    onboarding_status TEXT NOT NULL DEFAULT 'profile_required',
    rating            NUMERIC(3,2) DEFAULT 5.00,
    total_trips       INT  NOT NULL DEFAULT 0,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS provider_sessions (
    id                 UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider_id        UUID NOT NULL REFERENCES taxi_providers(id) ON DELETE CASCADE,
    refresh_token_hash TEXT NOT NULL,
    device_id          TEXT NOT NULL DEFAULT '',
    user_agent         TEXT NOT NULL DEFAULT '',
    ip_address         TEXT NOT NULL DEFAULT '',
    expires_at         TIMESTAMPTZ NOT NULL,
    revoked_at         TIMESTAMPTZ,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE IF NOT EXISTS vehicles (
    id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider_id   UUID NOT NULL REFERENCES taxi_providers(id) ON DELETE CASCADE,
    vehicle_class TEXT NOT NULL,
    plate_number  TEXT NOT NULL UNIQUE,
    seats         INT  NOT NULL DEFAULT 4,
    year          INT,
    make          TEXT,
    model         TEXT,
    color         TEXT,
    status        TEXT NOT NULL DEFAULT 'active',
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS taxi_bookings (
    id                 UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id        UUID NOT NULL,
    provider_id        UUID REFERENCES taxi_providers(id),
    vehicle_id         UUID REFERENCES vehicles(id),

    pickup_address     TEXT NOT NULL,
    pickup_lat         DOUBLE PRECISION NOT NULL,
    pickup_lng         DOUBLE PRECISION NOT NULL,

    dropoff_address    TEXT,
    dropoff_lat        DOUBLE PRECISION,
    dropoff_lng        DOUBLE PRECISION,
```

```
    vehicle_class      TEXT NOT NULL DEFAULT 'economy',
    passenger_count    INT  NOT NULL DEFAULT 1,
    ride_notes         TEXT NOT NULL DEFAULT '',

    distance_km        NUMERIC(8,2),
    duration_minutes   INT,
    fare_estimate_kobo BIGINT,
    fare_final_kobo    BIGINT,
    actual_distance_km NUMERIC(8,2),

    payment_method     TEXT NOT NULL DEFAULT 'wallet',
    payment_status     TEXT NOT NULL DEFAULT 'unpaid',
    payment_intent_id  TEXT,

    status             TEXT NOT NULL DEFAULT 'pending_match',
    cancel_reason      TEXT,
    cancelled_by       TEXT,

    matched_at         TIMESTAMPTZ,
    accepted_at        TIMESTAMPTZ,
    arrived_at         TIMESTAMPTZ,
    started_at         TIMESTAMPTZ,
    completed_at       TIMESTAMPTZ,
    cancelled_at       TIMESTAMPTZ,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE IF NOT EXISTS booking_events (
    id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL REFERENCES taxi_bookings(id) ON DELETE CASCADE,
    event_type TEXT NOT NULL,
    actor_type TEXT NOT NULL,
    actor_id   TEXT NOT NULL,
    metadata   JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS booking_reviews (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id        UUID NOT NULL UNIQUE REFERENCES taxi_bookings(id) ON DELETE CASCADE,
    customer_id       UUID NOT NULL,
    provider_id       UUID NOT NULL REFERENCES taxi_providers(id),
    rating            INT  NOT NULL CHECK (rating BETWEEN 1 AND 5),
    review_text       TEXT NOT NULL DEFAULT '',
    recommends_driver BOOLEAN,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_taxi_bookings_customer ON taxi_bookings(customer_id);
CREATE INDEX IF NOT EXISTS idx_taxi_bookings_provider ON taxi_bookings(provider_id);
CREATE INDEX IF NOT EXISTS idx_taxi_bookings_status   ON taxi_bookings(status);
CREATE INDEX IF NOT EXISTS idx_booking_events_booking ON booking_events(booking_id);
CREATE INDEX IF NOT EXISTS idx_vehicles_provider      ON vehicles(provider_id);
CREATE INDEX IF NOT EXISTS idx_provider_sessions_prov ON provider_sessions(provider_id);
```

`dropoff_address`, `dropoff_lat` and `dropoff_lng` are nullable. The supplied driver designs label the destination field "Drop off (optional)" on every screen. Section 8 defines fare handling for both cases.

Redis keys are namespaced under `taxi:`:

Key	Type	Purpose
`taxi:providers:online`	SET	Online driver ids

Key	Type	Purpose
`taxi:provider:status:{id}`	STRING, TTL = `PROVIDER_ONLINE_TTL`	Heartbeat lat/lng
`taxi:provider:matching:{id}`	STRING, SETNX with TTL	Per-driver match lock
`taxi:booking:rejected:{bookingID}`	SET with TTL	Drivers already offered this ride
`taxi:provider:auth:otp:{type}:{value}`	STRING	OTP challenge
`taxi:provider:auth:otp-rate:{key}`	STRING	OTP rate limit

`GetOnlineProviders` must prune members whose status key has expired. `SMEMBERS` returns stale ids.

6. Booking Lifecycle

```
pending_match → awaiting_acceptance → accepted → en_route_pickup
                                                        ↓
                                                arrived_at_pickup
                                                        ↓
                                                 in_progress → completed
```

Terminal states are `cancelled` and `unmatched`.

Status to driver screen mapping, from the supplied designs:

Status	Driver screen
`awaiting_acceptance`	Incoming Requests, Request Detail
`accepted`, `en_route_pickup`	Arriving at pick up in 1 minute
`arrived_at_pickup`	Arrived at Pickup; unlocks Start Trip
`in_progress`	Trip has started, with progress bar
`completed`	You have arrived, with belongings reminder

Current position

No lifecycle exists. The hauling service previously used read-then-write transitions and produced a defect where a driver accepted a booking and was then asked to accept it again.

Required behaviour

- Every transition is a guarded compare-and-set in SQL. Read-then-write is prohibited.
- A transition affecting zero rows returns `apperrors.Conflict`, never a 500 and never a silent success.
- Accept is guarded on both status and `provider_id`:

```
UPDATE taxi_bookings
   SET status='accepted', provider_id=$2, accepted_at=NOW(), updated_at=NOW()
 WHERE id=$1 AND status='awaiting_acceptance' AND provider_id=$2
RETURNING ...;
```

- Every provider endpoint verifies the booking's `provider_id` matches the token subject before acting.

- The customer may cancel through `arrived_at_pickup`. A cancel attempt at `in_progress` returns 409.
- The driver may cancel at any in-trip status.
- Every transition writes a `booking_events` row.

Intended result

Concurrent accept, reject and timeout paths resolve to one outcome, and no ride reaches a state the apps cannot render.

7. Matching Requirements

Current position

No matching exists. Matching runs as in-memory goroutines and does not survive a restart.

Required behaviour

- `CreateBooking` gates on `GetOnlineProviders`, not a raw `SCARD`. Zero eligible drivers rejects the booking rather than creating one that immediately becomes `unmatched`.
- Eligible drivers are filtered by haversine distance within `TAXI_MATCH_MAX_RADIUS_KM`, requested `vehicle_class`, sufficient `seats`, and absence from `taxi:booking:rejected:{id}`.
- Eligible drivers are sorted by ascending distance from pickup and offered one at a time.
- Each offer acquires a per-driver lock with TTL `MATCH_TIMEOUT + 15s`, performs a guarded `MarkMatched` from `pending_match` only, notifies the driver, and waits `MATCH_TIMEOUT`.
- On timeout the matcher attempts a guarded `ResetToMatching(id, expectedProviderID)`. If the reset is refused, the booking was accepted during the race window: matching stops and the lock is retained. Only a successful reset marks the driver rejected, releases the lock and advances to the next driver.
- A driver who times out is recorded in the rejected set. Without this, the nearest silent driver is re-offered on every rescan and consumes the whole window.
- Explicit reject cancels the in-flight match goroutine before re-dispatching. Only one matcher runs per booking.
- Matching stops when the driver list is exhausted or `TAXI_BOOKING_SEARCH_WINDOW`, measured from `created_at`, is spent. The booking is then marked `unmatched` by guarded compare-and-set and refunded.
- Live matchers are tracked in a `sync.Map` keyed by booking id so the recovery worker can skip them.

Intended result

Each driver is offered a given ride at most once, an accept is never overwritten by a timeout, and no booking is left searching after a restart.

8. Fare Requirements

Current position

No fare model exists. The `₦7.00` and `21 km · 8 min` values shown in the supplied designs are placeholder content.

Required behaviour

- All monetary values are kobo integers.
- The fare formula is:

```
fare = base_fare
     + (per_km_rate     × distance_km)
     + (per_minute_rate × duration_minutes)
     × vehicle_class_multiplier
rounded to the nearest 50 kobo, subject to a minimum fare.
```

- Where a drop-off is supplied, the fare is quoted before booking, stored in `fare_estimate_kobo`, and charged on driver acceptance.
- Where no drop-off is supplied, no quote is shown, no amount is held on acceptance, and the fare is computed at completion from `actual_distance_km` and stored in `fare_final_kobo`.
- Where the completed route differs from the quoted route, `fare_final_kobo` is recomputed at completion. A shortfall is charged and an excess is refunded against the held amount before settlement.
- The formula lives in its own package with pinned unit tests. If a partner surface is later added, the formula is promoted to `shared/go/taxipricing`, matching the treatment of `shared/go/dispatchpricing`.

Intended result

The passenger is charged for the journey actually taken, and the quoted and partner-facing fares cannot diverge.

9. Payment Requirements

Payment runs through `shared/go/walletclient` behind a local `PaymentClient` interface. Card and Paystack payment are excluded; wallet and cash only.

Event	Action	`payment_status`
Booking created, wallet	None	`unpaid`
Driver accepts, wallet	`CreatePaymentIntent` then `PayFromWallet`. A failure blocks the accept.	`held`
Driver accepts, cash	Record only	`unpaid`
Trip completed	Reconcile final fare, then `CompleteJob`	`paid`
Cancelled or unmatched	`RequestRefund`	`unpaid`

Requirements:

- `payment_intent_id` stores the wallet payment reference, not the intent UUID. Refunds resolve by reference or by the `source_service` and `source_reference` pair.
- `walletclient.AssignProvider(reference, "taxi", providerID)` is called when a driver is matched. `CompleteJob` refuses to settle without provider details, and settlement fails silently if this is omitted.
- `SourceService` is `taxi-service`, `SourceReference` is the booking id, `ProviderType` is `taxi`.
- Every wallet call carries an idempotency key: `taxi-hold-<bookingID>`, `taxi-pay-<bookingID>`, `refund-booking-<bookingID>`.

10. Notification Requirements

Current position

Taxi is absent from the notification service's trusted callers, per finding 3.2.

Required behaviour

- A `driver-taxi-service` block of event constants is added to `shared/go/notifications`. Hauling and dispatch event strings must not be reused; shared strings previously produced haulage copy on bike deliveries.
- Templates for every event are seeded in `services/notification-service/seeds/dev_templates.sql`. An event with no template and no inline title and body is rejected without error.

- Idempotency keys include the recipient: `bookingID + ":" + recipientID`. The notification service applies `ON CONFLICT (idempotency_key) DO NOTHING`, so a booking-only key drops the second and subsequent drivers' offers.

Required events:

Event	Recipient	Trigger
`ride.matched`	Driver	Ride offered
`ride.accepted`	Customer	Driver accepted
`ride.driver_en_route`	Customer	Driver departed for pickup
`ride.driver_arrived`	Customer	Driver at pickup
`ride.started`	Customer	Trip started
`ride.completed`	Customer	Trip ended
`ride.cancelled`	Both	Ride cancelled
`ride.unmatched`	Customer	No driver found

11. Endpoint Contracts

All paths are relative to `/api/v1/taxi`. Response envelopes follow the platform standard:

```
{ "success": true, "data": {} }
{ "success": false, "error": { "code": "validation_failed", "message": "...",
                               "request_id": "...", "fields": [] } }
```

Authentication uses `sharedauth.BearerMiddleware(signer, role, service)`. Customer routes verify role `customer` and service `customer` with `TAXI_CUSTOMER_TOKEN_SECRET`. Provider routes verify role `taxi_provider` and service `taxi` with `TAXI_PROVIDER_TOKEN_SECRET`. Internal routes use `serviceauth.Middleware`.

11.1 Public

Method	Path	Purpose
POST	`/customer/bookings/estimate`	Fare quote before booking

```
// request
{ "pickup_lat": 6.52, "pickup_lng": 3.37,
  "dropoff_lat": 6.43, "dropoff_lng": 3.42,
  "vehicle_class": "economy" }

// 200
{ "success": true, "data": {
    "distance_km": 12.4, "duration_minutes": 28,
    "fare_estimate_kobo": 385000,
    "breakdown": { "base_kobo": 100000, "distance_kobo": 248000,
                   "time_kobo": 37000, "class_multiplier": 1.0 } } }
```

11.2 Customer bookings

Method	Path	Purpose
GET	`/customer/availability?lat=&lng=`	Returns `{ available, count }`; gates the booking entry point
POST	`/customer/bookings`	Create a ride
GET	`/customer/bookings`	Ride history
GET	`/customer/bookings/:id`	Single ride; polled every 5 seconds
GET	`/customer/bookings/:id/location`	Driver's live position
PUT	`/customer/bookings/:id/cancel`	Cancel; 409 once `in_progress`
POST	`/customer/bookings/:id/review`	Rate the driver 1 to 5
GET	`/customer/bookings/:id/review`	Read an existing review
GET	`/customer/providers/:id`	Public driver profile
GET	`/customer/vehicles/:id`	Public vehicle detail

```
// POST /customer/bookings
{ "pickup_address": "24 Awawu str, Abaranje Ikotun Lagos",
  "pickup_lat": 6.52, "pickup_lng": 3.37,
  "dropoff_address": "23 Akeju Victoria Island",   // optional
  "dropoff_lat": 6.43, "dropoff_lng": 3.42,        // optional
  "vehicle_class": "economy",
  "passenger_count": 1,
  "ride_notes": "",
  "payment_method": "wallet" }

// 201 { "success": true, "data": { "booking": { } } }
// 400 no_drivers_available
// 422 validation_failed
```

`GET /customer/bookings/:id/location` returns 404 when the heartbeat key has expired. This is treated as location unavailable, not an error.

11.3 Provider authentication

Method	Path	Purpose
POST	`/provider/auth/start`	Phone to OTP challenge
POST	`/provider/auth/verify`	OTP to access and refresh tokens
POST	`/provider/auth/refresh`	Rotate session
POST	`/provider/auth/logout`	Revoke session

Requirements:

- Phone only. No email path. Phone numbers normalise through `shared/go/phonenumber` to `+234` format.
- OTP generation, hashing, token signing and session helpers come from `shared/go/auth`. The OTP hash binds the OTP secret, challenge id, typed identifier key and code.
- Access token claims are subject, role `taxi_provider`, service `taxi`, session id, token type, issued-at and expiry.

- Refresh tokens are opaque, prefixed by session id, stored only as HMAC hashes, and rotate on use. The previous session is revoked.
- When `TAXI_OTP_DEBUG=true`, the start response includes `debug_otp`. The Go struct field is tagged `json:"-"` and copied explicitly into the DTO. The value must never appear when the flag is false.

11.4 Provider profile and vehicles

Method	Path	Purpose
GET	`/provider/me`	Session bootstrap
GET	`/provider/profile`	Read profile
PUT	`/provider/profile`	Update profile
POST	`/provider/vehicles`	Register a vehicle
GET	`/provider/vehicles`	List own vehicles
GET	`/provider/vehicles/:id`	Read a vehicle
PUT	`/provider/vehicles/:id`	Update a vehicle

`PUT /provider/profile` uses preserve-on-empty semantics, `CASE WHEN $x != '' THEN $x ELSE col END`. Multiple provider screens submit disjoint field sets to this endpoint, and a plain update wipes the fields the current screen does not render.

`onboarding_status` moves from `profile_required` to `complete` on the first successful profile update.

11.5 Provider availability

Method	Path	Purpose
PUT	`/provider/availability`	Go online or offline
POST	`/provider/availability/heartbeat`	GPS ping every 30 seconds while online
GET	`/provider/availability`	Current state

Going online requires `onboarding_status='complete'` and at least one `active` vehicle. Each failure returns a distinct error code so the driver app can render the correct message.

11.6 Provider trip lifecycle

Method	Path	Transition
GET	`/provider/bookings`	List offers and active rides
GET	`/provider/bookings/:id`	Read one ride
PUT	`/provider/bookings/:id/accept`	`awaiting_acceptance` to `accepted`
PUT	`/provider/bookings/:id/reject`	`awaiting_acceptance` to `pending_match`
PUT	`/provider/bookings/:id/en-route-pickup`	`accepted` to `en_route_pickup`
PUT	`/provider/bookings/:id/arrived`	`en_route_pickup` to `arrived_at_pickup`

Method	Path	Transition
PUT	`/provider/bookings/:id/start`	`arrived_at_pickup` to `in_progress`
PUT	`/provider/bookings/:id/complete`	`in_progress` to `completed`
PUT	`/provider/bookings/:id/cancel`	Any in-trip status to `cancelled`
GET	`/provider/bookings/:id/review`	Passenger's rating
GET	`/provider/earnings`	Earnings summary and transaction list

`GET /provider/earnings` is a reporting projection over booking fares. It is not a ledger. The withdrawable balance is owned by payment-wallet-service, and the driver app reads its balance from there. The truck provider app previously displayed two different balances because this distinction was not observed.

11.7 Internal

Method	Path	Caller
GET	`/internal/providers/:id`	`support-dispute-service` identity snapshots

Returns `{ id, name, phone }`. Copy `services/driver-hauling-service/internal/features/identity/`. Required by finding 3.3.

11.8 Notification proxy

Method	Path
GET	`/provider/notifications`, `/customer/notifications`
POST	`/provider/notifications/devices`, `/customer/notifications/devices`
GET	`/provider/notifications/realtime-token`, `/customer/notifications/realtime-token`

Copy hauling's `internal/features/notifications/http`. Registration is skipped when `TAXI_NOTIFICATION_URL` is unset, and the apps fall back to polling.

12. Background Workers

Both workers sweep on start and then run on a ticker.

12.1 Match recovery, 30 second interval. Matchers are in-memory goroutines and a restart strands bookings in `pending_match` and `awaiting_acceptance` indefinitely. The worker queries stale rows, skips any with a live matcher, resumes matching where the search window remains, and otherwise marks the booking unmatched and refunds it.

12.2 Auto-complete, 5 minute interval, 30 minute grace. A failed completion call after a finished trip leaves the ride at `in_progress` with funds held and the driver unpaid. The worker completes those rides. The dispatch service shipped without this and it was raised as a P0.

13. Service Responsibility Boundary

This service must own	This service must not own
Taxi driver authentication, sessions and profiles	Customer identity, authentication and profile
Vehicle records and their verification state	Wallet ledger, balances and settlement arithmetic
Ride booking records and lifecycle state	Notification delivery infrastructure
Driver matching, offers and match locks	Media and document storage
Fare calculation and trip distance	Complaint and dispute records
Passenger ratings of drivers	Withdrawal processing and bank accounts
Trip earnings reporting projections	Authoritative provider payable balances

14. Inputs Required Before Build

The following values are owned by product and must be supplied before the sections named can be completed.

Input	Blocks
Base fare, per-kilometre rate, per-minute rate, minimum fare, in kobo	Section 8
Vehicle class list and per-class multipliers	Sections 5, 8, 11
Cancellation and no-show fee policy, including who is charged and at which status	Sections 6, 9
Maximum passenger count per vehicle class	Section 5
Whether pickup identity verification is in scope for this release	Sections 5, 6, 11

15. Delivery Sequence

Phase	Deliverable	Verification
0	Findings 3.1 to 3.4, `.env`, bootstrap script	Service boots; `/health` and `/api/v1/taxi/meta` respond
1	Config, database pool, migration runner, `001_taxi_core.sql`	Migration applies and is re-runnable
2	Provider authentication	Driver logs in by curl with `TAXI_OTP_DEBUG=true`
3	Provider profile and vehicles	Full CRUD; onboarding status advances
4	Availability and `/customer/availability`	Driver online returns `count: 1` to the customer
5	Fare package and booking CRUD	Estimates match pinned tests
6	Matching engine and recovery worker	Restart mid-offer recovers the booking
7	Lifecycle transitions	Full path walks; wrong-provider calls rejected
8	Payments, settlement, refunds	Wallet balances move correctly end to end

Phase	Deliverable	Verification
9	Notifications and templates	All eight events reach the recipient feed
10	Reviews, earnings, `/internal/providers/:id`	support-dispute resolves the driver name
11	Dev seeder and online helper	Local match testing without the apps

Phases 2 to 4 unblock the driver app. Phases 5 to 8 unblock the customer app.

The seeder follows `services/driver-dispatch-delivery-service/seeds/dev_seed.sql`. It seeds drivers in varied states, verified, onboarding-incomplete, suspended and unverified-vehicle, so every availability gate branch is reachable, plus completed, cancelled and in-progress rides. Fixed UUIDs with `ON CONFLICT DO NOTHING` keep it idempotent. All UUID literals must be valid hexadecimal.

16. Verification Requirements

`go build ./... && go vet ./... && go test ./...` must pass from the service directory.

Required tests:

- Fare formula, rounding and minimum fare, as pinned table tests.
- Matching: radius exclusion, class and seat filtering, dispatch to timeout to unmatched, and an accept-at-timeout-boundary case proving a late accept is not overwritten.
- Lifecycle: full path plus wrong-provider rejection at every step.
- Recovery: resume, expire, and skip-live-matcher.
- Payments: accept blocked on failed hold, settlement on completion, refund on cancel and unmatched.
- Handlers: missing token, wrong role and wrong service.

17. Known Failure Modes

Each of the following has already been shipped as a defect on this platform.

Failure	Prevention
Prefixed internal client URL	Bare origins only, section 4
HMAC service name not matching the callee's config key	Sign as `taxi-service`, section 3
Read-then-write status transitions	Guarded compare-and-set, section 6
Booking-only notification idempotency key	Include the recipient, section 10
Event-type strings shared across services	Own event block, section 10
High provider online TTL	90 seconds, section 4
`AssignProvider` omitted	Call on match, section 9
No auto-complete worker	Section 12.2
Unnormalised phone numbers	Normalise on write and on compare, section 11.3
Verification status seeded without underlying step rows	The app computes its badge from step records, section 15

18. Reference Implementations

Requirement	Source
Config shape, service bootstrap	`services/driver-hauling-service/internal/config/config.go`, `cmd/main.go`
Migration runner	`services/driver-hauling-service/internal/database/`
Provider OTP authentication	`services/driver-hauling-service/internal/features/provider_auth/`
Profile and vehicle CRUD	`services/driver-hauling-service/internal/features/provider_profile/`
Availability store	`services/driver-hauling-service/internal/features/provider_availability/`
Matching, lifecycle, recovery	`services/driver-hauling-service/internal/features/booking/usecases/booking_service.go`
Payment adapter	`services/driver-hauling-service/internal/features/booking/payments/`
Notification client and proxy	`services/driver-hauling-service/internal/features/booking/clients/`, `internal/features/notifications/http/`
Internal identity endpoint	`services/driver-hauling-service/internal/features/identity/`
Debug OTP in response	`services/driver-dispatch-delivery-service/internal/features/auth/`
Bootstrap script	`scripts/bootstrap_services/hauling-local-bootstrap.sh`
Dev seeder	`services/driver-dispatch-delivery-service/seeds/dev_seed.sql`

19. Expected Business Outcome

- A customer can book a car ride and be matched to a nearby driver within the search window.
- A driver receives, accepts and completes rides through the states the supplied designs render.
- Fares reflect the journey taken and settle to the driver's existing platform wallet.
- Ride history, ratings, notifications and complaints operate through the services that already own them.
- No booking, matching or trip state is duplicated outside this service.